

MSc thesis project ideas related to this course

Nils Anders Danielsson

2026-10-08

This course

In this course you have seen:

- ▶ Agda.
- ▶ Type checking.
- ▶ Equality checking.
- ▶ Normalisation (using reducibility).
- ▶ Abstract machines.

A formalisation

- ▶ An Agda formalisation of type theory:
<https://github.com/graded-type-theory/>.
- ▶ Authors: Abel, Danielsson, *Eriksson*, Favier, *Geng*, Gilbert, *Kubánek*, Nawrocki, *Öhman*, Vezzosi.

The type theory

Type formers: $\perp$, $\top$, Π , Σ , $\mathbb{N}$, $\equiv$, U , Level , $/$.

The type theory

Opaque definitions:

opaque

$f : A \rightarrow B$

$f = \dots$

The type theory

Erase:

$$\text{div} : (m\ n : \text{Nat}) \rightarrow @0\ n \not\equiv 0 \rightarrow \text{Nat}$$

The formalisation

Some theorems:

- ▶ Normalisation (using reducibility).
- ▶ Definitional equality is decidable.
- ▶ Type checking is decidable (for a certain class of terms).
- ▶ Erasure “works”.

Project ideas

`https://github.com/graded-type-theory/
graded-type-theory/labels/MSc`

Another project idea

- ▶ Agda compiles via GHC.
- ▶ However, a given Agda program can be considerably slower than a very similar Haskell program.
- ▶ Explore why, improve things, document.

Yet another project idea

Agda supports quotient types:

- ▶ Efficient data structures: Often multiple representations for a single “logical” value.
- ▶ Example: Balanced trees.
- ▶ Awkward for reasoning: Can use custom relation, but less convenient than builtin equality type.
- ▶ Can be addressed by quotient types.
- ▶ Example:

Finite-set : Set

Finite-set = Balanced-tree / Same-elements

Yet another project idea

- ▶ The quotient types use a newly invented interface using erasure.
- ▶ One case study: Finite sets as red-black trees quotiented by the “same elements” relation.
- ▶ Does this technique work more generally?

Questions? Contact me: `nad@cse.gu.se`.

Formalisation



Compiler



Quotients

